Webbasiertes Raumbuchungssystem mit Python

Konzeption, Implementierung und Qualitätssicherung

Portfolio zur Einzelabgabe im Modul Software Engineering

Studierender: Justin Kolar

Betreuer: Sandro Hartenstein

Anwendung: Raumzeit 1.1.0

Stand: 29. Oktober 2026

Vorgesehene Abgabe: 30. Oktober 2026

Abgabefassung mit dokumentierter KI-Unterstützung

Persönliche Funktionsprüfung und eigener Testlauf sind dokumentiert. Die Reflexion wurde KI-gestützt formuliert, von Justin Kolar persönlich geprüft, überarbeitet und am 29.10.2026 freigegeben. Der dokumentierte Prüfumfang bleibt maßgeblich.

Umfang: zwölf Seiten fachlicher Haupttext; Titelblatt, Verzeichnis und Quellenverzeichnis zusätzlich. Nachweise und Prompt-Log folgen im Anhang.

Inhaltsverzeichnis

1 Problemstellung und methodisches Vorgehen 3

2 Projektmanagement und Arbeitsorganisation 4

3 Anforderungen und Priorisierung 5

4 Akzeptanzkriterien und fachliche Regeln 6

5 Architektur und Verantwortlichkeiten 7

6 Designentscheidungen und SOLID 8

7 Datenmodell und HTTP-Schnittstelle 9

8 Umsetzung und Bedienablauf 10

9 Teststrategie und Auswahl der Fälle 11

10 Ausgeführte Tests und Messergebnisse 12

11 KI-Nutzung und kritische Quellenarbeit 13

12 Bewertung der Ergebnisse und Ausblick 14

Literatur und Quellenverzeichnis 15

Anhang

A Vollständige Anforderungen und Akzeptanzkriterien

B KI-Protokoll mit Originalauszügen aus Nutzereingaben

C Tatsächliche Browser- und Testnachweise

D Persönliche Prüfergebnisse, Reflexion und Belege

Hinweise zur Nachvollziehbarkeit

Die Kennungen F und N verbinden Anforderungen mit Python-Testnamen und den Browserfällen E01 bis E06. Die sechs Browserfälle laufen jeweils im Desktop- und Mobilviewport. Verwendet wird die numerische Zitierweise [1] bis [13]. Quellenabrufe erfolgten am 19. und 29.10.2026.

Dateibelege: output/tests/ enthält die aktuellen maschinenlesbaren Ergebnisse. docs/ enthält Auftrag, Projektverlauf, ADRs, Quellen, KI-Protokoll und persönliche Prüfergebnisse. Der persönliche Testlauf bleibt unter docs/nachweise separat erhalten. Quellstand und freigegebene Reflexion sind mit SHA-256-Prüfsummen abgesichert.

1 Problemstellung und methodisches Vorgehen

Gemeinsam genutzte Besprechungsräume benötigen eine eindeutige Zuordnung von Raum, Zeitpunkt und verantwortlicher Person. Werden Termine nur als frei bearbeitbare Liste geführt, können sich Zeiten überschneiden oder Änderungen unbemerkt gegenseitig überschreiben. Für diese Arbeit wird deshalb eine lokale Anwendung entwickelt, die Raumdaten und Buchungen verbindet und fachlich unzulässige Zustände bereits beim Speichern zurückweist. Die Zielgruppe ist eine Person, die die Raumbelegung an einem lokalen Arbeitsplatz koordiniert.

Die leitende Frage lautet: Wie lässt sich mit einer überschaubaren Python-Webanwendung eine widerspruchsfreie und nachvollziehbare Raumbuchung realisieren und durch ausgeführte Tests überprüfen? Das Ergebnis ist Raumzeit, eine Anwendung mit Wochenkalender, Raumverwaltung, Buchungsdialog und Stornierungshistorie. Erfolg bedeutet hier, dass die spezifizierten Geschäftsregeln erfüllt sind, Daten nach einem Neustart erhalten bleiben und typische Bedienabläufe nachvollziehbar funktionieren. Ein produktionsreifer, öffentlich zugänglicher Mehrbenutzerdienst ist kein Ziel dieser Fallstudie.

Methodisch folgt die Arbeit einer entwurfsorientierten Fallstudie mit anforderungsbasierter Verifikation. Aus dem vorgegebenen Problem werden zunächst Nutzerbedürfnisse, priorisierte Anforderungen und beobachtbare Akzeptanzkriterien abgeleitet. Darauf aufbauend werden Architekturentscheidungen dokumentiert, ein ausführbares Artefakt erstellt und die Kriterien durch Unit-, Integrations- und Browserprüfungen untersucht. Sollverhalten und tatsächliche Ergebnisse werden getrennt festgehalten. Aus den Ergebnissen werden Grenzen und Verbesserungen abgeleitet. Es wurden keine Nutzerinterviews oder empirischen Nutzungsstudien durchgeführt; die angenommene Zielgruppe ist daher eine Entwurfsannahme.

Die fachlichen Konzepte werden mit den Kursunterlagen [9] und Primärquellen abgeglichen. Die Priorisierung stützt sich auf MoSCoW [1], die Struktur auf getrennte Verantwortlichkeiten [2, 3] und die Verifikation auf mehrere Testgranularitäten [5, 8]. Der Zusammenhang zwischen Anforderungen, Umsetzung und Nachweisen ist über feste Kennungen hergestellt. Damit ist nicht allein das fertige Programm Gegenstand der Bewertung, sondern auch der dokumentierte Weg zu überprüfbaren Ergebnissen.

Die Arbeit entstand mit dokumentierter KI-Unterstützung. Der Studierende führte die praktischen Funktionsprüfungen durch, bewertete die Bedienbarkeit und startete die automatisierte Python-Suite selbst. Seine Reflexion wurde KI-gestützt formuliert, von ihm persönlich geprüft, überarbeitet und freigegeben. Kapitel 11 und Anhang D dokumentieren die Beiträge und den tatsächlichen Prüfumfang; die technische Umsetzung und die automatisierten Nachweise sind im KI-Protokoll zugeordnet.

2 Projektmanagement und Arbeitsorganisation

Das Einzelprojekt wird nach konkreten Ergebnissen und Abnahmekriterien organisiert. Die Planung vom 19.10.2026 bleibt als Baseline erhalten. Am 29.10.2026 wurde die vom Nutzer übermittelte unabhängige Bewertung zum Anlass einer erneuten Verifikation genommen. Die bisherige Suite war grün; zwei ergänzte Regressionstests bestätigten dennoch die gemeldeten Fehler. Damit wurde das Risiko unvollständiger Testauswahl tatsächlich sichtbar.

Tabelle 1: Zeitplan und tatsächlicher Stand der Überarbeitung

Arbeitspaket	Termin	Nachgewiesener Stand
Auftrag, Architektur und erste Anwendung	19.10.	Technische Arbeitsfassung erstellt
Geplante technische Meilensteine	22.–27.10.	Bereits am 19.10. erreicht; keine erfundenen Tätigkeiten an den Planterminen
Befunde reproduzieren und korrigieren	29.10.	Fehlerlauf gespeichert; technische und dokumentarische Überarbeitung
Persönliche Prüfung	29.10.	Bedienkontrollen und Testlauf dokumentiert; Reflexion geprüft, überarbeitet und freigegeben
PDF-Abgabe / Kolloquium	30.10. / danach	Einreichung offen; Gesprächstermin nicht belegt

Tabelle 2: Bewertungsmatrix und zugeordnete Nachweise

Bereich	Gewicht	Konkreter Beitrag
Projektmanagement	20 %	Baseline, Korrekturschleife, Risiken und dokumentierte persönliche Prüfung
Inhalt	50 %	Begründete Fachregeln, Versionsschutz, Migration und ausgeführte Tests
Struktur und Argumentation	20 %	Anforderung → Entscheidung → Test → tatsächliches Ergebnis
Formales	10 %	Einheitliche Zitate, beschriftete Darstellungen und visuelle PDF-Prüfung

Die wichtigsten Risiken sind gleichzeitige Buchungen, veraltete Dialoge und gemeinsam von KI-Code und KI-Tests übersehene Fehler. Gegenmaßnahmen sind atomare Transaktionen, Versionsprüfungen sowie zusätzliche Grenzfall- und Browserprüfungen. Testdatenbanken sind von Bestandsdaten getrennt. Verbleibende Grenzen sind der begrenzte persönliche Prüfumfang, ein einzelner getesteter Browser und eine nicht untersuchte hohe Mehrbenutzerlast.

Die Betreueranforderungen und Themenzustimmung sind nur durch die Nutzernachrichten belegt; es wird keine weitere Kommunikation behauptet. Es gibt keinen eingesehenen Moodle-Nachweis zusätzlicher Vorgaben. Der vollständige Projektverlauf benennt Plantermine, tatsächliche KI-Arbeiten und weiterhin offene Tätigkeiten getrennt.

3 Anforderungen und Priorisierung

MoSCoW unterscheidet zwingende, wichtige, optionale und ausgeschlossene Anforderungen [1]. Konsistenz und vollständige Buchungsabläufe sind für Raumzeit unverzichtbar; Suchkomfort folgt danach. Die Prioritäten sind begründete Entwurfsentscheidungen, keine Ergebnisse von Nutzerinterviews. Prozentwerte werden nicht aus der bloßen Anzahl der Anforderungen abgeleitet.

Tabelle 3: Funktionaler Umfang mit MoSCoW-Priorisierung

Kennung	Anforderung	Priorität / Stand
F01–F02	Räume pflegen; veraltete Änderungen abweisen	Must / umgesetzt
F03	Gültige Buchung anlegen	Must / umgesetzt
F04	Überschneidungen im selben Raum verhindern	Must / umgesetzt
F05–F06	Buchung ändern und stornieren	Must / umgesetzt
F07	Daten nach Neustart erhalten	Must / umgesetzt
F08	Arbeitswoche im Kalender anzeigen	Must / umgesetzt
F09–F11	Raumfilter, Suche und Stornierungshistorie	Should / umgesetzt
F12	Getrennte Beispieldaten für die Vorführung	Could / umgesetzt
C01–C02	ICS-Export und Serienbuchungen	Could / zurückgestellt
W01–W02	Anmeldung, Cloudbetrieb und Mailversand	Won't / ausgeschlossen

Anhang A enthält die vollständigen Kriterien und Testzuordnungen. Die Überarbeitung ergänzt F02 um Raumversionen, N04 um ungültige Token, N05 um kurze Termine und N07 um Datenmigration.

Tabelle 4: Nichtfunktionale Anforderungen und messbare Nachweise

ID	Qualitätsziel	Prüfkriterium
N01	Validierung / Must	Ungültige Angaben werden serverseitig ohne Speicherung abgewiesen
N02	Konsistenz / Must	Von zwei konkurrierenden Doppelbuchungen wird genau eine akzeptiert
N03	Antwortzeit / Should	Lokale Wochenabfrage mit 1.000 Buchungen: empirisches P95 unter 250 ms
N04	Lokaler Schutz / Must	CSRF, Hostprüfung und parametrisierte SQL-Zugriffe
N05	Bedienbarkeit / Should	390-Pixel-Viewport ohne horizontalen Seitenüberlauf; mobile Agenda
N06	Reproduzierbarkeit / Must	Dokumentierte Installation und maschinenlesbare Testergebnisse
N07	Datenkompatibilität / Must	Schema v1 additiv auf v2 migrieren; Altbestand und bestehende Versionen erhalten

Die Nachweise gelten für den dokumentierten lokalen Aufbau; mobile Tests belegen keine vollständige Barrierefreiheit.

4 Akzeptanzkriterien und fachliche Regeln

Given-When-Then beschreibt Ausgangslage, Handlung und beobachtbare Folge [9] und begründet die Testauswahl. Buchungen bilden halboffene Intervalle [Beginn, Ende): Ein Folgetermin kann unmittelbar anschließen.

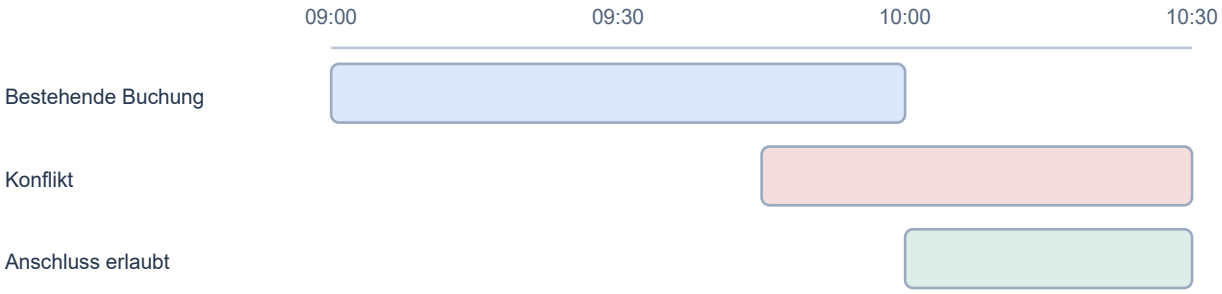


Abbildung 1: Halboffene Intervalle und Entscheidung über eine Überschneidung

Tabelle 5: Ausgewählte Akzeptanzfälle für F02 bis F06

Gegeben	Wenn	Dann
Raum frei; Kapazität 12	Sechs Personen buchen 09:00–10:00	Eine aktive Buchung wird gespeichert
Aktiv 09:00–10:00	Gleicher Raum 09:45–10:15	Konflikt; keine zusätzliche Buchung
Aktiv 09:00–10:00	Gleicher Raum 10:00–11:00	Angrenzender Termin wird akzeptiert
Raum bietet 12 Plätze	13 Personen werden eingetragen	Validierungsfehler; Daten bleiben unverändert
Buchung wurde inzwischen geändert	Alte Version wird gespeichert	HTTP 409; aktueller Stand bleibt erhalten
Termin wird storniert	Derselbe Zeitraum wird erneut gebucht	Neuer Termin möglich; Historie bleibt erhalten
Raumdialog mit Version 1	Inzwischen besteht Version 2	HTTP 409; Stand von Version 2 bleibt erhalten

Im selben Raum besteht eine Überschneidung, wenn bestehender Beginn < neues Ende und bestehendes Ende > neuer Beginn gilt. Nur aktive Buchungen zählen; bei Bearbeitung wird die eigene ID ausgeschlossen. Gleiche Startzeiten allein würden Teilüberschneidungen übersehen.

Buchungen liegen Montag bis Freitag zwischen 08:00 und 20:00 Uhr am selben Tag im 15-Minuten-Raster. Die positive ganze Personenzahl darf die Kapazität nicht überschreiten. Vergangene Termine sind zur nachträglichen Dokumentation zulässig. Zeitangaben sind lokale Berliner Wandzeit; Zeitzonenwechsel und Kalendersynchronisation bleiben ausgeschlossen.

Nachweiskette F02: Das Kriterium verbietet verlorene Raumänderungen; ADR 04 begründet den Versionsvergleich. test_stale_room_edit_http_preserves_current_record und E05 prüfen 409 sowie den unveränderten aktuellen Datensatz. Die Fehlerantwort allein würde den Datenerhalt nicht belegen.

5 Architektur und Verantwortlichkeiten

Raumzeit ist ein modularer Monolith. Die Browseroberfläche und der lokale Python-Prozess bilden die Laufzeitumgebung; SQLite speichert die Daten in einer Datei. Fachliche Regeln, Ablaufsteuerung, HTTP und Persistenz sind getrennte Verantwortlichkeiten. Das folgende Diagramm zeigt sowohl den Laufzeitfluss als auch den entscheidenden Unterschied zur Richtung der Quellcodeabhängigkeiten.

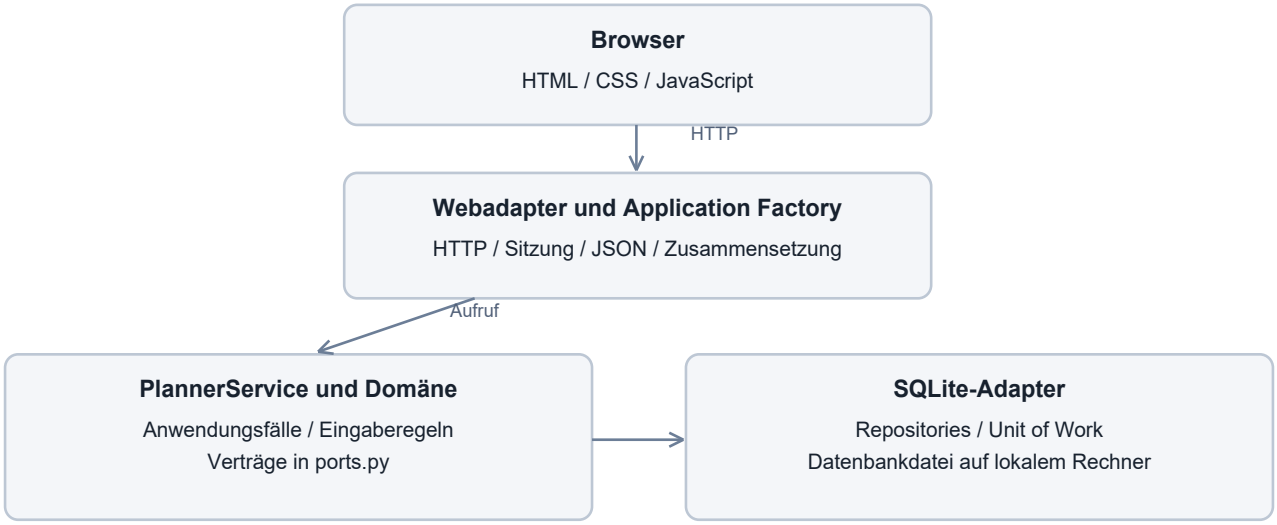


Abbildung 2: Komponenten und Abhängigkeiten des lokalen Systems

Die Domäne enthält Datenklassen und fachliche Eingaberegeln. PlannerService koordiniert die Anwendungsfälle und verwendet dafür Verträge aus ports.py. Diese Protokolle beschreiben Raum- und Buchungsoperationen sowie den Rahmen einer gemeinsamen Transaktion. Der SQLite-Adapter implementiert die benötigten Operationen; die Webschicht setzt Daten in JSON um und behandelt Sitzung, CSRF und HTTP-Fehler. Die Application Factory erzeugt die konkrete Zusammensetzung und erlaubt getrennte Konfigurationen für regulären Betrieb, Demo und Tests [4].

Martins Dependency Rule verlangt, dass innere fachliche Bereiche keine äußeren technischen Details kennen [2]. Hier importieren domain.py und application.py weder Flask noch sqlite3. Trotzdem kann der Service zur Laufzeit die Datenbank verwenden, weil ihm die UnitOfWorkFactory übergeben wird. Die statische Abhängigkeit richtet sich somit auf einen fachlichen Vertrag. Die Architektur übernimmt diese Trennung, ohne ein vollständiges mehrschichtiges Framework nachzubauen.

Der SQLite-Adapter verantwortet zusätzlich die additive Schema-Migration. Das hält Datenbanksyntax aus dem Service heraus. Ein ORM oder eine verteilte Bereitstellung würde hier weitere Abhängigkeiten schaffen, ohne die Kernfrage zu lösen. Die bewusste Grenze des Entwurfs ist ein lokaler Arbeitsbereich; ein zweiter vollständiger Speicheradapter wurde nicht umgesetzt.

6 Designentscheidungen und SOLID

Die Designentscheidungen sind in sechs Architecture Decision Records mit Kontext, Alternativen und Konsequenzen festgehalten. SOLID dient dabei als Prüfperspektive für nachvollziehbare Verantwortlichkeiten und Verträge [3]. Nicht jedes Prinzip wird allein durch die Verwendung von Klassen oder Interfaces erfüllt; insbesondere wird eine noch nicht erprobte Austauschbarkeit nicht als erwiesenes Ergebnis dargestellt.

Tabelle 6: SOLID-Bezug am implementierten Entwurf

Prinzip	Konkrete Anwendung und Grenze
SRP	HTTP, Fachabläufe und SQL liegen in getrennten Modulen. PlannerService bündelt die eng zusammengehörigen lokalen Planungsfälle; bei stark wachsendem Umfang wäre eine weitere Aufteilung zu prüfen.
OCP	Ein anderer Speicheradapter könnte über die Factory ergänzt werden. Neue Geschäftsregeln benötigen weiterhin gezielte Änderungen an Domäne oder Service; vollständige Änderungsfreiheit wird nicht behauptet.
LSP	Ein Adapter muss Bedeutung und Fehlerverhalten der Repository-Verträge erhalten, insbesondere Transaktionskonsistenz. Es gibt hier nur einen vollständigen Adapter; dessen allgemeine Austauschbarkeit ist nicht mit einem zweiten Adapter nachgewiesen.
ISP	Raum- und Buchungszugriffe haben getrennte, kleine Protokolle. Ein universelles Repository mit ungenutzten CRUD-Funktionen wird vermieden.
DIP	Die Anwendungslogik hängt von UnitOfWorkFactory und Repository-Verträgen ab. Die Webschicht injiziert den konkreten SQLite-Adapter beim Zusammensetzen.

Tabelle 7: Zentrale Entscheidungen und Abwägungen

ADR	Entscheidung	Alternative und Konsequenz
01	Flask und SQLite, lokal	Django oder ein verteilter Aufbau böten weitere Infrastruktur; der lokale Umfang bleibt einfacher vorführbar.
02	BEGIN IMMEDIATE; halboffene Intervalle	Frontendprüfung oder UNIQUE auf Beginn reichen nicht. Schreibzugriffe werden serialisiert; hohe Last bleibt offen.
03	Repositories und Unit of Work	SQL in HTTP-Routen wäre kürzer, würde aber fachliche Prüfung und technische Zugriffe vermischen.
04–06	Versionen, Migration und Regressionen	Alte Dialogstände liefern 409; alte Datenbankdateien bleiben mit Schema v2 nutzbar; fehlerhafte Token werden vor dem Vergleich abgewiesen.

Die Bewertung zeigt die Grenze einer rein strukturellen Betrachtung: Getrennte Module und eine hohe Abdeckung verhinderten den CSRF-Randfall nicht. Der zulässige Eingabetyp des verwendeten Vergleichs [10] musste zusätzlich geprüft werden. Ebenso löst eine Schreibsperre kein länger geöffnetes, veraltetes Dialogformular. Transaktion und gelesene Versionsnummer sichern unterschiedliche Zeiträume ab.

7 Datenmodell und HTTP-Schnittstelle

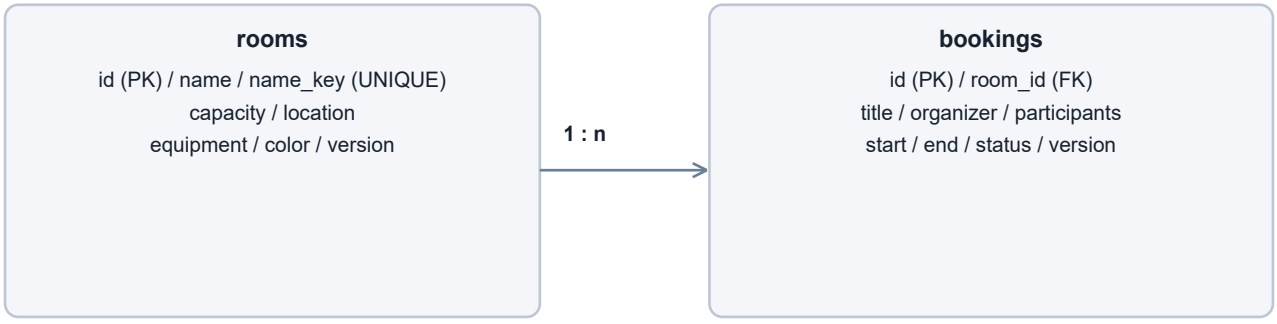


Abbildung 3: Relationales Datenmodell mit Raumbezug und Versionsfeldern

Buchungen referenzieren Räume über Fremdschlüssel; beide besitzen eine positive version. Constraints prüfen elementare Werte, der Service fachliche Zusammenhänge. Die additive Migration ergänzt rooms.version mit DEFAULT 1 und setzt PRAGMA user_version auf 2 [11]. Wiederholtes Initialisieren erhält Versionen; ein unbekanntes neueres Schema führt zum Rollback.

Tabelle 8: Öffentliche lokale HTTP-Schnittstellen

Methode und Pfad	Bedeutung	Erfolg
GET /api/config	Grenzen und Metadaten lesen	200
GET /api/rooms	Räume lesen	200
POST /api/rooms	Raum anlegen	201
PUT /api/rooms/{id}	Raum mit zuvor gelesener Version ändern	200
GET /api/bookings?from=…&to=…	Zeitraum lesen; optional room_id	200
POST /api/bookings	Buchung anlegen	201
PUT /api/bookings/{id}	Buchung mit gelesener Version ändern	200
POST /api/bookings/{id}/cancel	Buchung mit Version stornieren	200

Änderungen erwarten JSON und ein CSRF-Token. GET liefert die aktuelle Version, die der Dialog bei PUT mitsendet. Ohne version folgt 422, bei veraltetem Stand 409. Bestehende Datenbankdateien bleiben verwendbar; alte API-Clients müssen das Versionsfeld ergänzen.

```
{"room_id": "UUID", "title": "Projektbesprechung",
"organizer": "Test Person", "participants": 6,
"start": "2026-10-19T09:00",
"end": "2026-10-19T10:00", "version": 1}
```

Nicht-ASCII-Token werden vor compare_digest mit 403 abgewiesen [10]; gültige Token funktionieren weiter. Weitere Fehlercodes: 422 bei Validierung, 404 bei fehlenden Daten, 409 bei Konflikten, 400 bei ungültigem JSON und 503 mit Retry-After bei Datenbankfehlern.

8 Umsetzung und Bedienablauf

Die Oberfläche stellt die Arbeitswoche als Kalender dar. Raumfarben erleichtern die Orientierung; ein Raumfilter reduziert die sichtbaren Termine. Parallel liegende Termine unterschiedlicher Räume erhalten nebeneinander liegende Kalenderpositionen. Auf schmalen Bildschirmen wird stattdessen eine chronologische Agenda angezeigt. Alle Ressourcen liegen lokal; eine externe Schrift- oder Grafikabfrage ist für den Betrieb nicht erforderlich.

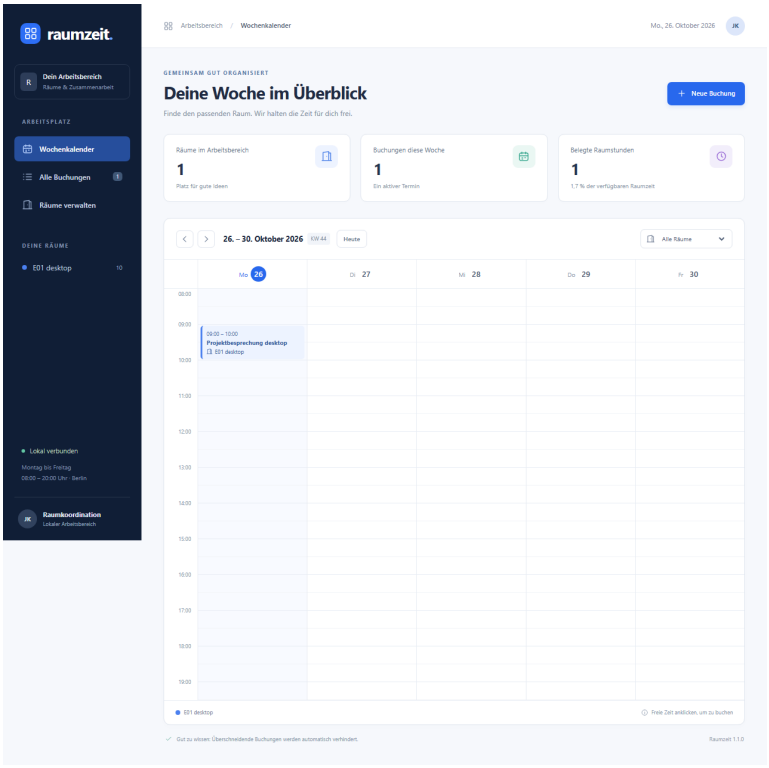


Abbildung 4: Wochenkalender aus dem reproduzierbaren Desktop-Test E01; Browserdatum auf den 26.10.2026 fixiert, echte HTTP-Anwendung

Eine neue Buchung öffnet einen Dialog mit Titel, Raum, Personenzahl, verantwortlicher Person, Datum und Uhrzeit. Ein Klick auf eine freie Kalenderposition übernimmt den Tag und eine passende Startzeit. Die Formularprüfung unterstützt die Eingabe, ersetzt aber nicht die serverseitige Validierung. Nach erfolgreichem Speichern werden Wochenansicht, Liste und Kennzahlen neu geladen. Eine Konfliktmeldung lässt den Dialog geöffnet, damit der Zeitraum korrigiert werden kann.

Raum- und Buchungsdialog senden ihre gelesene Version. Nach einem Konflikt bleibt das Formular geöffnet und fordert zum Schließen und Neuladen auf. Ein 15-Minuten-Termin erhält im Desktopkalender nur eine Titelzeile; Zeit und Raum bleiben über Tooltip und zugänglichen Namen verfügbar. Mobil stellt die Agenda den Termin ohne die enge Kalenderhöhe dar. E04 prüft Textgeometrie, Tastaturöffnung und die korrekten Zeitfelder.

Die bestehende HTML-, CSS- und JavaScript-Struktur wurde ohne Frameworkwechsel mit einer fixierten Prettier-Version lesbar formatiert. Ein wiederholbarer Formatcheck kontrolliert diesen Zustand. Für Browsertests gelten package-lock.json und Playwright 1.64.1; für die Dokumentausgabe separate, festgelegte Python-Pakete einschließlich ReportLab und Pillow. Weder Browser- noch PDF-Werkzeuge sind Laufzeitabhängigkeiten des Raumplaners.

9 Teststrategie und Auswahl der Fälle

Die Verifikation kombiniert isolierte Eingaberegeln, echte SQLite-Integration, HTTP-Tests und Browserassertionen. Die Testpyramide [8] wird risikoorientiert angewendet: Für verlorene Änderungen und atomare Konfliktprüfung zählt das tatsächliche Verhalten der Datenbank. Ein Mock, der immer erfolgreich schreibt, könnte diese Frage nicht beantworten.

Tabelle 9: Ausgeführte automatisierte Testfälle nach Ebene

Ebene	Fälle	Zweck
Domäne	37	Isolierte Eingaberegeln und Grenzwerte
Anwendungsfälle + SQLite	42	Konsistenz, Versionen, Migration und Konkurrenz
HTTP-Adapter + Regressionen	39	Testclient, JSON, CSRF-Formate und Statuscodes

Grenzwertfälle prüfen Öffnungszeiten, positive ganze Personenzahlen und erlaubte Intervalle. Integrationstests starten unabhängige Verbindungen mit synchronisiertem Beginn. Bei konkurrierender Buchung wird genau ein Erfolg erwartet; bei zwei Änderungen mit derselben Raumversion ebenfalls genau ein Erfolg und eine stale-Ablehnung. Der aktuelle Datensatz wird anschließend vollständig verglichen.

Tabelle 10: Konkrete Kette von Anforderung bis Nachweis

Anforderung	Entscheidung	Test und kontrolliertes Ergebnis
F02 Raumänderung	ADR 04 / 05	Alter Dialog: 409; Version 2 und aktuelle Felder bleiben erhalten
N04 CSRF	ADR 06	Unicode-Header: 403 auf allen Änderungsrouten; gültiges Token: 201
N07 Altbestand	ADR 05	Schema v1 → v2; Buchungen unverändert; Neustart idempotent
N05 kurzer Termin	ADR 06	E04: sichtbare Titelzeile, Tastaturöffnung und richtige Uhrzeiten

Sechs Playwright-Szenarien laufen jeweils bei 1280 × 900 und 390 × 844 Pixeln: Anlegen/Neuladen, Überschneidung, Bearbeiten/Stornieren, Kurztermin, Raumdialog-Konflikt sowie Navigation/Filter/Suche. Der Testserver nutzt eine neue SQLite-Datei und keine API-Mocks [12]. Nur der Browser-Datumswert ist für den Kalender fixiert; Timer und reale Requests bleiben aktiv [13]. Screenshots ergänzen die Assertions. Python-Abdeckung misst ausdrücklich keinen JavaScript-Code. E06 legt Vergleichsraum und beide Buchungen selbst an und wird zusätzlich einzeln auf frischen Datenbanken geprüft.

10 Ausgeführte Tests und Messergebnisse

Die folgende Auswertung wird beim PDF-Bau aus den tatsächlich erzeugten Berichten geladen. JUnit, Coverage, Browserreport und Prüfmanifeste gehören zu demselben Quellstand. Der ursprüngliche Abgabestand bleibt getrennt archiviert. Dadurch sind historische Befunde und aktuelle Ergebnisse unterscheidbar.

Tabelle 11: Tatsächlicher Verifikationsstand vom 29.10.2026

Nachweis	Wert	Einordnung
Python-Testfälle	118	Alle bestanden; keine übersprungenen Fälle
Python-Zeilen	367 / 370	99.19 %
Python-Zweige	75 / 76	98.68 %
Python-Abdeckung kombiniert	99.10 %	Keine Aussage zur JavaScript-Abdeckung
Browserausführungen	12	Sechs Szenarien in zwei Viewports; alle bestanden
Umgebung	Python 3.12.14	Flask 3.1.3; SQLite 3.53.1

Am 29.10. bestand zunächst die unveränderte Suite mit 88 Tests. Die beiden neuen Regressionsfälle scheiterten anschließend wie gemeldet: Token ä erzeugte einen TypeError, und die veraltete Raumänderung wurde mit 200 angenommen. review-before.xml bewahrt diese Befunde. Nach der Korrektur wurden alle Python-Fälle sowie die Browserfälle erneut ausgeführt. Zusätzlich wurden zwei vorhandene Datenbankdateien als isolierte Kopien zweimal initialisiert; ihre Originale blieben unverändert. Die weitere Abnahme deckte trotz zwölf erfolgreicher Browserausführungen eine Testabhängigkeit auf: E06 benötigte fremde Buchungen. Nach eigenem Fixture-Aufbau bestanden der Desktop- und Mobil-Einzellauf sowie erneut die gesamte Browsersuite (e06-before.xml, e06-desktop-alone.xml, e06-mobile-alone.xml).

Tabelle 12: Lokale Adaptermessung mit 1.000 Buchungen

Messgröße	Ergebnis
Median	9.335 ms
Empirisches 95. Perzentil	10.32 ms
Maximum	14.377 ms
Zielwert / Ergebnis	P95 < 250 ms / erfüllt

Die Adaptermessung verwendet zehn Räume und 1.000 Buchungen, fünf Aufwärmläufe und 100 Testclient-Abfragen einschließlich JSON-Dekodierung. P95 ist der 95. sortierte Messwert. Sie erfasst weder Browserdarstellung noch Netzlatenz oder Dauerlast. Die Abdeckung betrifft Python einschließlich Zweigen; der CLI-Start ist ausgenommen. Bestandene Tests und hohe Abdeckung belegen die ausgewählten Fälle, keine allgemeine Fehlerfreiheit.

11 KI-Nutzung und kritische Quellenarbeit

Der KI-Assistent erstellte Architektur-, Implementierungs-, Test- und Dokumententwürfe und führte technische Prüfungen aus. Der Studierende übernahm die praktische Funktionsprüfung, die Bewertung der Bedienbarkeit und einen eigenen Start der Python-Suite. Seine Reflexion ist persönlich geprüft, überarbeitet und freigegeben. Anhang B belegt die KI-Nutzung, Anhang D den persönlichen Prüfumfang. Zusätzliche Betreuerkommunikation wird nicht behauptet.

Tabelle 13: KI-Arbeitsschritte und angewandte Überprüfung

KI-Ergebnis	Prüfung	Verbleibende Grenze
Anforderungen	Abgleich mit Mail, Bewertungsmatrix und beobachtbaren Kriterien	Kein Interview oder empirisch bestätigter Bedarf
Architektur und Code	Versionen, Migration, echte SQLite-Integration und Konkurrenzfälle	Kein zweiter vollständiger Adapter
Testentwürfe	Ausführung; Fehlerbefunde erhalten und Korrekturen erneut geprüft	KI erzeugte auch die Tests
Oberfläche	Wiederholbare Playwright-Assertions in Chromium bei zwei Viewports	Keine vollständige Browser- oder Screenreadermatrix
Theorie und Text	Hersteller- und Originalquellen; Aussagen auf gemessenen Aufbau begrenzt	Persönliche Reflexion freigegeben; Quellenprüfung nicht vollständig belegt

Die Literatur wird nach ihrem Beitrag unterschieden. MoSCoW [1] begründet den Umgang mit einem festen Termin, liefert aber keine projektspezifische Priorisierung. Diese muss aus der Kernfrage abgeleitet werden. Architekturquellen [2, 3] geben Kriterien zur Trennung von Verantwortlichkeiten; daraus folgt nicht automatisch die Notwendigkeit einer großen Frameworkstruktur. Für die konkrete Sperrwirkung einer Transaktion ist die Herstellerbeschreibung von SQLite [6] die maßgebliche technische Quelle. Diese unterschiedlichen Geltungsbereiche werden in den Designentscheidungen zusammengeführt.

Auch die Testliteratur wird angepasst: Die Pyramidenidee [8] unterstützt mehrere Ebenen, ersetzt jedoch nicht die Analyse des größten Risikos. Da Konflikte zwischen zwei Schreibvorgängen datenbankspezifisch sind, sind reale Integrationstests hier aussagekräftiger als zusätzliche isolierte Mocks. Die Sicherheitsdokumentation [7] dient zur Auswahl lokaler Schutzmaßnahmen, nicht als Zertifikat für eine öffentliche Bereitstellung. Die Umsetzung bleibt ausdrücklich auf den lokalen Arbeitsbereich begrenzt.

Der grüne Ausgangslauf mit 88 Fällen und die danach reproduzierten Fehler veranschaulichen ein gemeinsames Blindfeld von Implementierung und Tests. Die später entdeckte E06-Abhängigkeit zeigt zudem, warum ein erfolgreicher Gesamtlauf die Isolation einzelner Tests nicht belegt. Konkrete Gegenbeispiele und isolierte Wiederholungen verringern dieses Risiko. Die persönlichen Kontrollen ergänzen diese Nachweise, ersetzen jedoch keine vollständige unabhängige Codeprüfung.

12 Bewertung der Ergebnisse und Ausblick

Die Kernfrage wird für den spezifizierten lokalen Aufbau beantwortet: Raumzeit verwaltet Raum- und Buchungsdaten und erzwingt die getesteten Konsistenzregeln. Entscheidend ist nicht die Kalenderdarstellung allein, sondern das Zusammenspiel von Eingaberegeln, atomarer Schreibtransaktion und eindeutigem Konfliktverhalten. Benachbarte Intervalle bleiben möglich, untersuchte Überschneidungen werden abgewiesen.

Die Überarbeitung ergänzt zwei konkret belegte Lücken. Erstens werden ungültige Unicode-Token kontrolliert behandelt. Zweitens schützt eine gelesene Versionsnummer nun auch Raumdaten vor dem Überschreiben aus älteren Dialogen. Die additive Migration ermöglicht die Weiterverwendung vorhandener Datenbankdateien. Der Browserfall mit zwei gleichzeitig geöffneten Formularen zeigt den Nutzen des Versionsschutzes über einen rein sequenziellen API-Test hinaus.

Python-Tests prüfen Fachregeln, Persistenz und HTTP-Grenzen; Playwright führt JavaScript im echten Chromium aus und kontrolliert Bedienabläufe. Die lokale P95-Messung bewertet die Adapterantwort. Die persönliche Erprobung des Studierenden ergänzt dies um konkrete Bedienrückmeldungen. Sie ist keine breit angelegte Nutzungsstudie und belegt ebenso wenig wie die automatisierten Tests eine Sicherheit für öffentliche Bereitstellung.

Der Umfang bleibt ein lokaler Arbeitsbereich ohne Anmeldung, Rollen, Cloudbetrieb oder Serienbuchungen. Die verantwortliche Person ist ein Textfeld. Zeitangaben sind Berliner Wandzeit ohne externe Zeitzonenumrechnung. Stornierungen bleiben erhalten, aber frühere Inhaltsstände werden nicht als unveränderliches Auditlog archiviert. Es gibt keine automatische Datensicherung. Ein Downgrade auf den alten Anwendungscode ist nicht geprüft und wird nicht unterstützt.

Der Studierende bestätigte die Buchungs-, Stornierungs-, Neustart- und Raumkonfliktprüfungen und startete die Python-Suite selbst: 118 Fälle bestanden. Er bewertete die Oberfläche als benutzerfreundlich und berichtete keine Unklarheiten oder Überraschungen. Daraus formulierte er Erweiterungsvorschläge für passende Raumempfehlungen und das Wiederherstellen stornierter Termine. Seine persönlich überarbeitete und freigegebene Reflexion ist unverändert in Anhang D übernommen; die dokumentierten Prüfgrenzen bleiben erhalten.

Die vorgesehene Abgabe ist der 30.10.2026. Der etwa 15-minütige Gesprächstermin wird anschließend mit dem Betreuer vereinbart; ein bestätigter Termin liegt hier nicht vor. Die fachliche Vorbereitung konzentriert sich auf begründete Entscheidungen, die tatsächlichen Prüfungen und eine zutreffende Darstellung der KI-Unterstützung.

Literatur und Quellenverzeichnis

[1] Agile Business Consortium (2026). What is MoSCoW Prioritization?. https://www.agilebusiness.org/resource/what-is-moscow-prioritization/ (Zugriff: 19.10.2026).

[2] Martin, Robert C. (2012). The Clean Architecture. https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html (Zugriff: 19.10.2026).

[3] Martin, Robert C. (2020). Solid Relevance. https://blog.cleancoder.com/uncle-bob/2020/10/18/Solid-Relevance.html (Zugriff: 19.10.2026).

[4] Pallets Projects (o. J.). Application Factories. Flask Documentation 3.1.x. https://flask.palletsprojects.com/en/stable/patterns/appfactories/ (Zugriff: 19.10.2026).

[5] Pallets Projects (o. J.). Testing Flask Applications. Flask Documentation 3.1.x. https://flask.palletsprojects.com/en/stable/testing/ (Zugriff: 19.10.2026).

[6] SQLite Developers (o. J.). Transaction. https://www.sqlite.org/lang_transaction.html (Zugriff: 19.10.2026).

[7] Pallets Projects (o. J.). Security Considerations. Flask Documentation 3.1.x. https://flask.palletsprojects.com/en/stable/web-security/ (Zugriff: 19.10.2026).

[8] Vocke, Ham (2018). The Practical Test Pyramid. https://martinfowler.com/articles/practical-test-pyramid.html (Zugriff: 19.10.2026).

[9] Hartenstein, Sandro (2025). Software Engineering. Anforderungen und SoftwareArchitektur Basics. Bereitgestellte Kursunterlagen im Ordner Folien PDF-20261019; Anforderungen, Folien 20–26; Architektur Basics, Folien 16–18.

[10] Python Software Foundation (o. J.). hmac.compare_digest. Python 3.12 Documentation. https://docs.python.org/3.12/library/hmac.html#hmac.compare_digest (Zugriff: 29.10.2026).

[11] SQLite Developers (o. J.). ALTER TABLE. https://www.sqlite.org/lang_altertable.html (Zugriff: 29.10.2026).

[12] Microsoft Playwright (o. J.). Web server. Playwright Documentation. https://playwright.dev/docs/test-webserver (Zugriff: 29.10.2026).

[13] Microsoft Playwright (o. J.). Clock. Playwright Documentation. https://playwright.dev/docs/clock (Zugriff: 29.10.2026).

Die bereitgestellten Unterlagen von 2025 werden für Fachkonzepte verwendet. Die Einzelabgabe 2026 richtet sich nach den im Chat übermittelten Vorgaben und der Bewertungsmatrix; diese Kommunikation ist Projektgrundlage und keine veröffentlichte Fachliteratur.

Abbildungen und Tabellen sind eigene Darstellungen des vorliegenden Projekts. Screenshots zeigen die tatsächlich ausgeführte Anwendung mit fiktiven Beispieldaten.

Anhang A Anforderungen und Akzeptanzkriterien

Die folgende Spezifikation ist aus docs/anforderungen.json übernommen. Die als umgesetzt ausgewiesenen Anforderungen besitzen konkrete Kriterien und zugeordnete Nachweise. Could- und Won't-Einträge grenzen spätere Wünsche ab.

F01 Räume anzeigen

Priorität: Must | Stand: umgesetzt

Als koordinierende Person möchte ich Räume mit Kapazität und Ausstattung sehen, damit ich einen geeigneten Ort wählen kann.

AK 1: Given gespeicherte Räume, When die Raumansicht geöffnet wird, Then werden Name, Standort, Kapazität und Ausstattung angezeigt.

AK 2: Given keine Räume, When die Raumansicht geöffnet wird, Then erscheint eine verständliche Aufforderung zum Anlegen eines Raums.

Nachweise: test_configuration_and_empty_state; E05

F02 Räume anlegen und ändern

Priorität: Must | Stand: umgesetzt

Als koordinierende Person möchte ich Räume pflegen, damit ihre Daten für Buchungen stimmen.

AK 1: Given ein vollständiger eindeutiger Raumname und eine Kapazität von 1 bis 500, When gespeichert wird, Then bleibt der Raum nach erneutem Abruf erhalten.

AK 2: Given eine aktive Buchung mit sechs Personen, When die Raumkapazität auf fünf reduziert wird, Then wird die Änderung ohne Datenverlust abgelehnt.

AK 3: Given ein mit Version 1 geöffneter Raumdialog, When bereits Version 2 gespeichert wurde, Then wird die alte Änderung mit HTTP 409 abgewiesen und Version 2 unverändert erhalten.

Nachweise: test_room_edit_http; test_room_edit_protects_booked_capacity; test_duplicate_room_name_unicode_casefold; E05; test_stale_room_edit_http_preserves_current_record; test_concurrent_room_edits_only_one_succeeds

F03 Buchung anlegen

Priorität: Must | Stand: umgesetzt

Als koordinierende Person möchte ich einen freien Raum für einen Termin buchen, damit die Belegung feststeht.

AK 1: Given ein freier Raum mit ausreichender Kapazität, When ein gültiger Termin gespeichert wird, Then entsteht genau eine aktive Buchung mit Version 1.

AK 2: Given fehlende Pflichtfelder, ungültige Zeiten oder eine zu große Gruppe, When gespeichert wird, Then werden Fehler verständlich angezeigt und keine Daten geändert.

Nachweise: test_create_and_persist; test_invalid_booking_rejected; test_capacity_exactly_allowed_and_excess_rejected; E01

F04 Doppelbuchungen verhindern

Priorität: Must | Stand: umgesetzt

Als koordinierende Person möchte ich überschneidende Termine im selben Raum verhindern, damit die Raumplanung widerspruchsfrei bleibt.

AK 1: Given eine aktive Buchung 09:00–10:00, When 09:45–10:15 für denselben Raum gebucht wird, Then wird der Konflikt ohne zusätzliche Buchung abgewiesen.

AK 2: Given dieselbe Ausgangslage, When 10:00–11:00 gebucht wird, Then wird der angrenzende Termin akzeptiert.

AK 3: Given ein freier Zeitraum, When zwei unabhängige Verbindungen gleichzeitig buchen, Then wird genau eine Buchung angelegt.

Nachweise: test_overlap_rejected_without_partial_write; test_adjacent_bookings_allowed; test_concurrent_requests_only_one_succeeds; E02; E01

F05 Buchungen ändern

Priorität: Must | Stand: umgesetzt

Als koordinierende Person möchte ich Termine bearbeiten, damit Änderungen nicht zu neuen oder widersprüchlichen Buchungen führen.

AK 1: Given eine aktive Buchung, When sie mit aktueller Version geändert wird, Then bleibt die ID gleich und die Version steigt um eins.

AK 2: Given einen zwischenzeitlich geänderten Datensatz, When eine ältere Version gespeichert wird, Then wird HTTP 409 zurückgegeben und der aktuelle Datensatz erhalten.

Nachweise: test_edit_excludes_itself_and_increments_version; test_edit_conflict_rolls_back; test_stale_update_cannot_overwrite; E03

F06 Buchung stornieren

Priorität: Must | Stand: umgesetzt

Als koordinierende Person möchte ich einen Termin stornieren, damit der Raum erneut genutzt werden kann.

AK 1: Given eine aktive Buchung, When die Stornierung bestätigt wird, Then wird ihr Status cancelled und der Zeitraum wieder buchbar.

AK 2: Given eine stornierte Buchung, When sie erneut mit aktueller Version storniert wird, Then entstehen weder ein weiterer Datensatz noch eine weitere Versionsänderung.

Nachweise: test_cancel_frees_slot_and_retains_history; test_stale_cancel_rejected; E03

F07 Daten dauerhaft speichern

Priorität: Must | Stand: umgesetzt

Als koordinierende Person möchte ich gespeicherte Daten nach einem Neustart wiederfinden, damit die Planung erhalten bleibt.

AK 1: Given eine gespeicherte Buchung, When eine neue Serviceinstanz dieselbe Datenbank öffnet, Then ist die unveränderte Buchung vorhanden.

AK 2: Given eine fehlgeschlagene Konfliktprüfung, When erneut abgefragt wird, Then ist ausschließlich der zuvor gültige Stand gespeichert.

Nachweise: test_create_and_persist; test_overlap_rejected_without_partial_write

F08 Wochenkalender bedienen

Priorität: Must | Stand: umgesetzt

Als koordinierende Person möchte ich die Belegung einer Arbeitswoche sehen, damit ich freie Zeitfenster erkennen kann.

AK 1: Given Termine der ausgewählten Woche, When der Kalender geladen wird, Then werden aktive Buchungen von Montag bis Freitag mit Raum und Uhrzeit angezeigt.

AK 2: Given die Kalenderansicht, When vorwärts oder rückwärts navigiert wird, Then ändert sich der abgefragte Wochenzeitraum; Heute führt zur aktuellen Woche.

Nachweise: test_range_is_end_exclusive_and_room_filter_works; E06

F09 Nach Raum filtern

Priorität: Should | Stand: umgesetzt

Als koordinierende Person möchte ich die Anzeige auf einen Raum begrenzen, damit ich dessen Belegung schnell erkenne.

AK 1: Given Termine mehrerer Räume, When ein Raum ausgewählt wird, Then erscheinen nur dessen Kalendertermine.

AK 2: Given ein gesetzter Filter, When Alle Räume gewählt wird, Then erscheinen wieder alle aktiven Termine der Woche.

Nachweise: test_range_is_end_exclusive_and_room_filter_works; E06

F10 Buchungen durchsuchen

Priorität: Should | Stand: umgesetzt

Als koordinierende Person möchte ich Titel und verantwortliche Personen durchsuchen, damit ich einen Termin gezielt wiederfinde.

AK 1: Given mehrere Buchungen, When ein Teil des Titels oder Personennamens eingegeben wird, Then erscheinen passende Einträge unabhängig von Groß- und Kleinschreibung.

AK 2: Given keinen Treffer, When gesucht wird, Then erscheint ein verständlicher Leerzustand.

Nachweise: E06

F11 Stornierungen nachvollziehen

Priorität: Should | Stand: umgesetzt

Als koordinierende Person möchte ich stornierte Termine gesondert sehen, damit ihr Verbleib nachvollziehbar bleibt.

AK 1: Given eine stornierte Buchung, When die Historie gefiltert wird, Then bleibt sie mit dem Status Storniert sichtbar.

AK 2: Given eine Stornierung, When der Wochenkalender angezeigt wird, Then blockiert die Buchung keinen Zeitraum mehr.

Nachweise: test_cancel_frees_slot_and_retains_history; E03

F12 Getrennte Beispieldaten

Priorität: Could | Stand: umgesetzt

Als vorführende Person möchte ich eine getrennte Demo starten, damit ich die Anwendung ohne echte Daten zeigen kann.

AK 1: Given eine neue Demo-Datenbank, When --demo gestartet wird, Then werden vier Räume und zwölf gültige Termine der aktuellen Woche angelegt.

AK 2: Given eine bestehende Demo, When erneut gestartet wird, Then werden keine zusätzlichen Beispieldatensätze angelegt und die reguläre Datenbank bleibt unberührt.

Nachweise: test_demo_is_separate_and_idempotent

C01 Kalenderexport

Priorität: Could | Stand: zurückgestellt

Ein Export als ICS wäre für externe Kalender nützlich.

AK 1: Für eine spätere Version zu spezifizieren; in dieser Abgabe nicht umgesetzt.

C02 Serienbuchungen

Priorität: Could | Stand: zurückgestellt

Wiederkehrende Termine wären für regelmäßige Veranstaltungen nützlich.

AK 1: Für eine spätere Version einschließlich Serienkonflikten zu spezifizieren; in dieser Abgabe nicht umgesetzt.

W01 Anmeldung und Rollen

Priorität: Won't | Stand: ausgeschlossen

Benutzerkonten und berechtigungsabhängige Änderungen sind außerhalb des lokalen Einzelarbeitsplatzes.

AK 1: In dieser Version nicht enthalten; keine behauptete Benutzeridentität oder Zugriffstrennung.

W02 Cloudbetrieb und Mailversand

Priorität: Won't | Stand: ausgeschlossen

Veröffentlichung und externe Benachrichtigungen werden in dieser Abgabe nicht benötigt.

AK 1: Kein Hosting und kein E-Mail-Versand im Anwendungsumfang.

N01 Verständliche Validierung

Priorität: Must | Stand: umgesetzt

Fehleingaben sollen keine ungültigen Datensätze erzeugen.

AK 1: Pflichtfelder, positive ganze Personenzahl, Kapazitätsgrenze, Werktag, gleicher Tag und 15-Minuten-Raster werden serverseitig geprüft.

AK 2: Validierungsfehler liefern HTTP 422 und eine deutschsprachige Meldung; Überschneidungen liefern 409.

Nachweise: test_invalid_booking_rejected; test_validation_and_unknown_record_statuses; E02

N02 Atomare Konsistenz

Priorität: Must | Stand: umgesetzt

Prüfen und Schreiben müssen auch bei Konkurrenz eine konsistente Belegung erhalten.

AK 1: Genau eine von zwei konkurrierenden Buchungen desselben Raums und Zeitraums wird akzeptiert.

AK 2: Fehlgeschlagene Änderungen lassen den bisherigen Datensatz unverändert.

Nachweise: test_concurrent_requests_only_one_succeeds; test_edit_conflict_rolls_back

N03 Lokale Antwortzeit

Priorität: Should | Stand: gemessen

Die Wochenabfrage soll bei kleinen lokalen Datenbeständen zügig antworten.

AK 1: Bei 1.000 Buchungen, zehn Räumen, fünf Aufwärmläufen und 100 Testclient-Abfragen liegt das empirische 95. Perzentil unter 250 ms.

AK 2: Die Messung wird ausdrücklich als lokale Adaptermessung ohne Netz und Browser dokumentiert.

Nachweise: benchmark.json

N04 Lokaler Schutz

Priorität: Must | Stand: umgesetzt

Unbeabsichtigte Änderungen durch fremde Webseiten und einfache Eingabeangriffe sollen erschwert werden.

AK 1: Alle unterstützten Änderungsrouten erfordern ein sitzungsgebundenes CSRF-Token; fremde Hostnamen werden abgewiesen.

AK 2: SQL-Parameter und textContent behandeln eingegebene Inhalte als Daten; ein öffentliches Hosting wird nicht gestartet.

AK 3: Given ein nicht-ASCII-haltiges oder falsches CSRF-Token, When eine Änderungsroute aufgerufen wird, Then folgt kontrolliert HTTP 403; mit gültigem Token bleibt eine gültige Änderung möglich.

Nachweise: test_mutations_require_csrf; test_untrusted_host_blocked; test_sql_parameterization_preserves_literal_data; test_non_ascii_csrf_is_forbidden; test_unicode_csrf_rejected_on_every_mutation; test_valid_csrf_still_allows_creation

N05 Mobile Bedienbarkeit

Priorität: Should | Stand: geprüft

Die Kernansichten sollen auch bei schmaler Browserbreite nutzbar bleiben.

AK 1: Bei einem 390-Pixel-Viewport ersetzt eine Agenda den engen Wochenkalender und es gibt keinen horizontalen Seitenüberlauf.

AK 2: Der Buchungsdialog ist sichtbar und die Felder haben Beschriftungen; eine vollständige WCAG-Konformität wird nicht behauptet.

AK 3: Given ein 15-Minuten-Termin, When er im Kalender angezeigt wird, Then ist eine Titelzeile ohne vertikales Abschneiden sichtbar und der Dialog per Tastatur erreichbar; mobil wird der Termin in der Agenda geöffnet.

Nachweise: E01 (mobile); E04 (desktop/mobile)

N06 Reproduzierbare Ausführung

Priorität: Must | Stand: umgesetzt

Die technische Prüfung soll mit dokumentierten Befehlen und festgelegten Abhängigkeiten wiederholbar sein.

AK 1: README, festgelegte Python-Pakete und Startskripte beschreiben die lokale Einrichtung.

AK 2: tools/verify.py erzeugt tatsächliche JUnit- und Coverage-Ergebnisse in isolierten Testdatenbanken.

AK 3: Browserprüfung und PDF-Erstellung besitzen getrennte dokumentierte Abhängigkeiten und wiederholbare Befehle; Python-Abdeckung wird nicht als Browserabdeckung ausgegeben.

Nachweise: junit.xml; environment.json; requirements.txt; requirements-portfolio.txt; package-lock.json; browser_source_manifest.json

N07 Vorhandene Datenbanken weiterverwenden

Priorität: Must | Stand: geprüft

Vorhandene Raum- und Buchungsdaten sollen mit dem Versionsschutz weiter nutzbar bleiben.

AK 1: Given ein Schema-v1-Datenbestand ohne Raumversion, When die neue Anwendung initialisiert, Then werden Raumversion 1 und Schema-Version 2 ergänzt, ohne bestehende Buchungen oder Raumfelder zu verlieren.

AK 2: Given eine bereits migrierte Datenbank, When erneut initialisiert wird, Then bleiben Daten und vorhandene Versionsnummern erhalten. Eine unbekannte neuere Schema-Version wird nicht herabgesetzt.

Nachweise: test_legacy_migration_preserves_data_and_is_idempotent; test_newer_schema_not_downgraded; migration.json

Anhang B Dokumentation der KI-Nutzung

Stand: 29.10.2026; ursprüngliche Bearbeitung am 19.10.2026. Werkzeug: OpenAI Codex im Chat. Eine genaue Modell- oder Produktversionskennung wurde für diesen Nachweis nicht unabhängig erfasst. Die KI war an Planung, Quellcode, Tests, Recherche und Dokumentation beteiligt. KI-gestützte technische Läufe und die persönlichen Funktionsprüfungen des Studierenden werden getrennt dokumentiert. Seine Reflexion ist persönlich geprüft, überarbeitet und freigegeben; Nachweise stehen in Anhang D.

Prompt-Log mit Originalauszügen

Die nachstehenden Zitate sind tatsächlich im verfügbaren Gespräch enthaltene Nutzereingaben. Verkürzungen sind als Auszug gekennzeichnet. Dies ist ein kuratiertes fachliches Prompt-Log und kein unveränderter technischer Export sämtlicher Chat- und Werkzeugnachrichten. Für einen vollständigen Gesprächsnachweis ist vor Abgabe zusätzlich der Originalchat zu sichern. Die folgende Dokumentation enthält keine erfundenen Zwischenprompts.

P01

Originaleingabe des Studierenden: „ich möchte erstmal mit dir das vorgehen und die herangehensweise planen schreibe mir bitte ebenfalls eine kurze antwort mail“ (Auszug)

Ergebnis und verwendete Teile: Unterlagen ausgewertet, Anforderungen zusammengefasst, E-Mail-Entwurf erstellt

Nachweis und Prüfung: Auftrag gegen übermittelte Mail geprüft; kein E-Mail-Versand

P02

Originaleingabe des Studierenden: „du machst das doch sowieso also ist das doch egal - aber ich mag python am meisten“ (Antwort im Planungsdialog)

Ergebnis und verwendete Teile: Python als bevorzugte Sprache festgehalten

Nachweis und Prüfung: Umfang und Erklärung der Architektur auf eine kleine Anwendung begrenzt

P03

Originaleingabe des Studierenden: „also lager und nachbestellung ist wahrscheinlich zu nah am ehemaligen projekt - aber genaueres zum ablauf und möglichen themenvorschlägen können wir ja in die mail hauen und dann soll hartenstein sagen was wir machen oder?“

Ergebnis und verwendete Teile: Neues Thema; zwei Vorschläge für die Betreuerabstimmung

Nachweis und Prüfung: Kein Quellcode oder Text aus dem Gruppenbericht übernommen

P04

Originaleingabe des Studierenden: „hartenstein meinte beide themen sind super also entscheide du welches cooler wäre“

Ergebnis und verwendete Teile: Raumbuchungssystem ausgewählt

Nachweis und Prüfung: Zustimmung des Betreuers liegt hier nur als Bericht des Studierenden vor

P05

Originaleingabe des Studierenden: „alles klar du hast die anforderungen was da alles drin sein muss und wie die projektdoku oder so aussehen soll also kanns losgehen“ (Auszug; anschließend vollständige Bewertungsmatrix)

Ergebnis und verwendete Teile: Umsetzung von Anwendung, Tests und Portfolio; Matrix in die Struktur aufgenommen

Nachweis und Prüfung: Quellenabgleich, Akzeptanztests, echter Browser, dokumentierter Fehlerlauf und Wiederholung

P06 Überarbeitung nach unabhängiger Bewertung am 29.10.2026

Originalauszug: „Bitte prüfe die Befunde zunächst selbst und setze die technischen und dokumentarischen Verbesserungen anschließend um“. Der Auftrag umfasst CSRF, Raumversionen, Browsertests, Lesbarkeit, Abhängigkeiten und Portfolio;

vollständiger Wortlaut: docs/review_auftrag.md.

Ergebnis und Prüfung: Der Assistent führte zuerst die vorhandene Suite aus und reproduzierte anschließend beide Defekte mit neuen, zunächst fehlgeschlagenen Regressionstests. Danach ergänzte er ASCII-Prüfung, Raumversionen, Migration, lesbare Frontendformatierung und wiederholbare Playwright-Fälle in zwei Viewports. Neue Zahlen stammen aus den aktuellen Berichten; alte Ergebnisse bleiben erkennbar archiviert. Die vorhandenen Datenbanken wurden für eine zusätzliche Migrationsprüfung nur lesend geöffnet und ihre Kopien verändert.

Zu diesem Zeitpunkt war die persönliche Prüfung noch offen; die Aufgaben P1–P6 bereiteten sie vor. Die späteren tatsächlichen Rückmeldungen und die Freigabe sind in P08 und Anhang D dokumentiert.

P07 Abgabefassung und persönliche Prüfung vorbereiten am 29.10.2026

Originalauszug: „Leite mich anschließend schrittweise durch P1–P6. Gib mir konkrete Bedienaufgaben und Verständnisfragen und warte auf meine tatsächlichen Beobachtungen und Antworten.“ Vollständige Anweisung: docs/abschlussauftrag.md.

Ergebnis und Prüfung: Der Assistent reproduzierte den E06-Fehler einzeln auf frischer Datenbank (e06-before.xml), ergänzte eigene Vergleichsdaten und prüfte E06 danach separat in Desktop und Mobil (e06-desktop-alone.xml, e06-mobile-alone.xml). Anschließend bestanden alle zwölf Browserausführungen. Der P3-Sollzustand wurde auf den stornierten ersten Termin und den weiterhin aktiven Anschlusstermin präzisiert; die ADR-Anzahl wurde auf sechs korrigiert. Der Abgabeabgleich steht in docs/abgabecheck.md. Danach führte der Studierende die persönlichen Kontrollen durch und gab die überarbeitete Reflexion frei.

P08 Persönliche Überarbeitung und Freigabe am 29.10.2026

Originalauszug: „Ich habe die persönliche Reflexion selbst überarbeitet. Bitte verwende die aktuell gespeicherte Fassung als meine freigegebene Endfassung und ändere ihren Inhalt nicht eigenständig.“ Außerdem: „Aktualisiere den Vorspann und die Statusangaben zutreffend: KI-gestützt formuliert, von mir persönlich geprüft und überarbeitet.“

Ergebnis und Prüfung: Der Assistent las und sicherte die gespeicherte Datei, aktualisierte ausschließlich Vorspann und Statusüberschrift und übernahm den freigegebenen Text samt Grenzabschnitt unverändert. Inhaltsprüfsummen und PDF-Textvergleich sichern diese Übernahme ab. Die persönlichen Bedienrückmeldungen und der selbst gestartete 118-Test-Lauf sind getrennt von den KI-gestützten Abschlussläufen dokumentiert. Die zusätzliche persönliche Code-Durchsicht blieb ausgelassen. Es werden keine weiteren Tätigkeiten oder Lernerfahrungen ergänzt.

Vom Assistenten gewählte und ausgeführte Arbeitsschritte

Diese Schritte sind Arbeitsaufzeichnungen des Agenten, keine nachträglich erfundenen Nutzereingaben:

- Lokalen modularen Monolithen entworfen, Repository-Verträge und Unit of Work eingesetzt; Alternativen in ADRs begründet.

- Zeitregeln, Kapazitätsprüfung, Status und Versionsprüfung implementiert.

- Kalender, Buchungsdialog, Raumansicht, Filter und Historie erstellt; keine externen Gestaltungsassets oder kostenpflichtigen Dienste eingesetzt.

- Unit- und Integrationstests aus fachlichen Gegenbeispielen erstellt, ausgeführt und tatsächliche Fehler behoben. Die erste Ausführung mit vollständig nutzbarer Testumgebung ergab 84 bestandene und vier fehlgeschlagene Tests. Das frühere JUnit-Ergebnis ist erhalten.

- Quellen der Hersteller beziehungsweise Originalautoren geöffnet und mit Implementierungsaussagen verglichen. Die verwendeten Quellen sind in docs/quellen.json nachgewiesen.

- Lokalen Browserablauf einschließlich Konflikt und Stornierung ausgeführt. Diese Prüfung ist keine vom Studierenden ausgeführte manuelle Abnahme.

- Dokumententwurf aus Projektdateien und Messwerten erstellt. Testzahlen werden aus den Ergebnisdateien übernommen, nicht frei formuliert.

Wie Ergebnisse geprüft wurden

Anforderungen wurden zuerst als beobachtbare Bedingungen formuliert. Tests prüfen vollständige und partielle Überschneidungen, benachbarte Intervalle, Grenzwerte der Öffnungszeit, Kapazität, fehlende Datensätze, Persistenz,

Stornierung, veraltete Versionen und Konkurrenz. SQL-Zugriffe werden gegen echte temporäre SQLite-Dateien geprüft. HTTP-Tests prüfen Statuscodes und CSRF. Die Browserprüfung ergänzt die serverseitigen Tests um tatsächliche Darstellungen und Bedienhandlungen.

Die KI erzeugte auch die Tests. Deshalb ist ein gemeinsamer Denkfehler weiterhin möglich. Die hohe gemessene Python-Abdeckung sagt nicht aus, dass die Spezifikation vollständig oder das JavaScript vollständig geprüft ist. OCP/LSP werden nicht mit einem erfundenen zweiten Adapter belegt. Performance wird ausschließlich im gemessenen lokalen Aufbau beschrieben. Diese Grenzen sind im Portfolio enthalten.

Persönliche Prüfung und Freigabe

Justin Kolar hat die praktischen Bedienkontrollen zurückgemeldet und die Python-Suite selbst gestartet. Die Reflexion wurde von ihm persönlich überarbeitet und ausdrücklich freigegeben. Sein dokumentierter Prüfumfang und dessen Grenzen stehen in Anhang D; die Rohbelege des persönlichen Testlaufs bleiben separat erhalten. Die Einreichung und der Abgleich nicht vorliegender zusätzlicher Moodle-Vorgaben bleiben persönliche Schritte.

Anhang C Ausgeführte Browser- und Testnachweise

KI-gestützt erstellte und ausgeführte Playwright-Tests mit echtem Chromium, Flask und SQLite. Dieser automatisierte Browsernachweis ist von der Python-Abdeckung und den persönlichen Kontrollen in Anhang D getrennt. Das Browserdatum ist auf den 26.10.2026 fixiert; die tatsächliche Ausführung erfolgte am 29.10.2026.

E01 Raum und Buchung über die UI erstellen, nach Neuladen sichtbar; mobil Agenda ohne horizontalen Überlauf.

Ist: desktop: passed (968 ms); mobile: passed (1030 ms)

E02 Teilüberschneidung liefert 409 und sichtbare Meldung; vollständiger alter Datensatz bleibt erhalten.

Ist: desktop: passed (1126 ms); mobile: passed (1093 ms)

E03 Änderung liefert Version 2; Stornierung Version 3 und Historieneintrag, dieser Termin ist nicht mehr aktiv.

Ist: desktop: passed (1257 ms); mobile: passed (1440 ms)

E04 15-Minuten-Titel ohne vertikales Abschneiden; Tastatur öffnet Dialog mit den richtigen Zeiten.

Ist: desktop: passed (1088 ms); mobile: passed (1014 ms)

E05 Zwei Raumdialoge mit Version 1: erster speichert Version 2, zweiter erhält 409; aktueller Datensatz unverändert.

Ist: desktop: passed (1077 ms); mobile: passed (981 ms)

E06 Wochenwechsel/Heute, Raumfilter sowie Suche nach Titel und Person einschließlich Leerzustand funktionieren.

Ist: desktop: passed (1725 ms); mobile: passed (1892 ms)

Die Testdatei prüft außerdem auf unbehandelte JavaScript-Ausnahmen. Chromium in zwei Viewports ersetzt keine vollständige Browser- und Screenreadermatrix. Erwartungswerte sind Assertions, keine erfundenen persönlichen Beobachtungen.

Anhang C Bildschirmnachweise

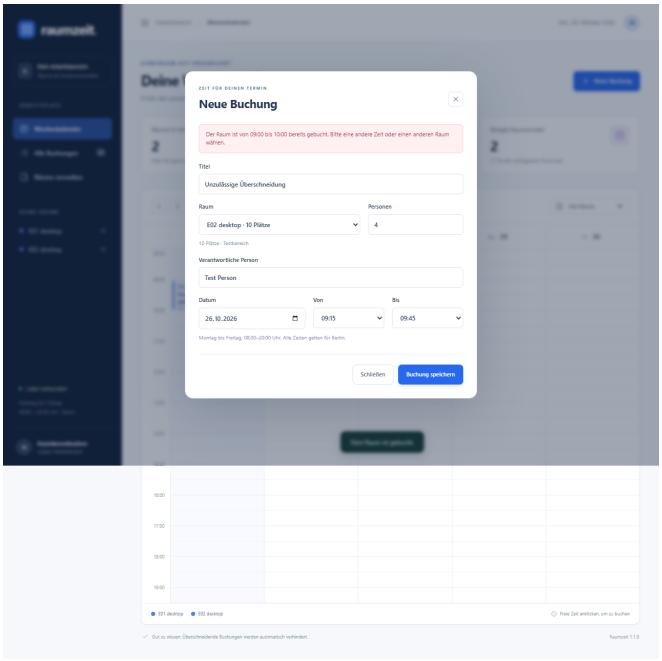


Abbildung 5: Zurückweisung einer Überschneidung im wiederholbaren Browserfall E02

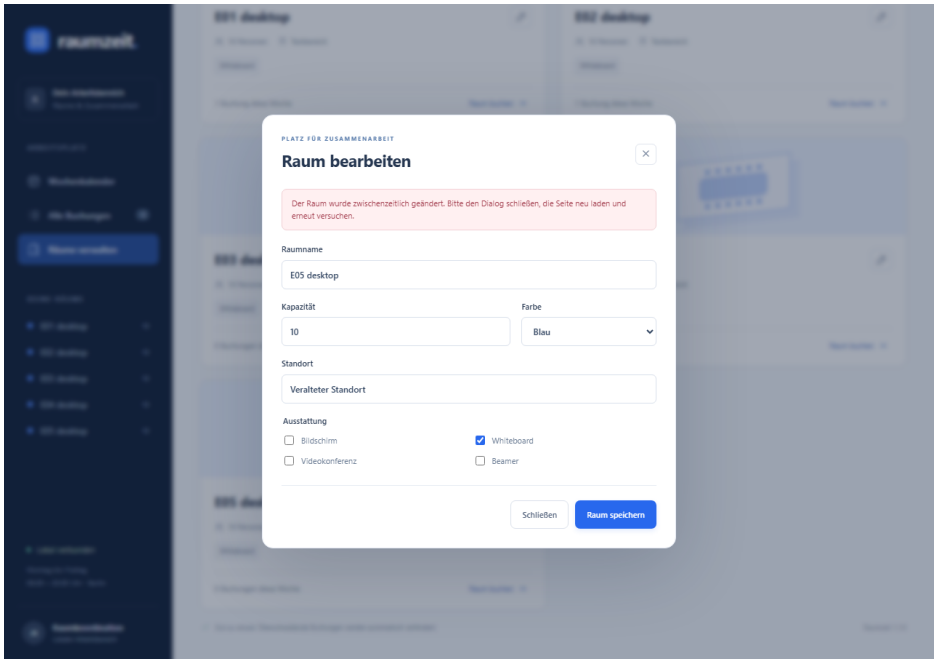


Abbildung 6: Veralteter Raumdialog wird nach vorheriger Änderung mit 409 abgewiesen; Fall E05

Anhang C Mobile Darstellung und Ergebnisdateien

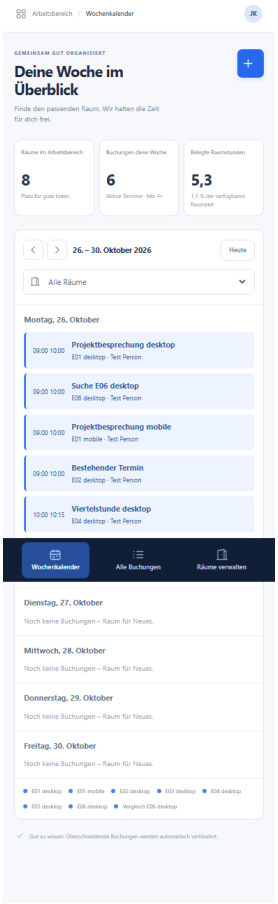


Abbildung 7: Mobile Agenda im 390-Pixel-Viewport; Browserfall E01

Nachweisdateien

output/tests/junit.xml; coverage.json; environment.json

output/tests/review-before.xml (zwei reproduzierte Fehler)

output/tests/e06-before.xml; e06-desktop-alone.xml; e06-mobile-alone.xml

output/tests/benchmark.json; migration.json

output/playwright/junit.xml; results.json; screenshots/

output/tests/browser.json; source_manifest.json; browser_source_manifest.json

JUnit und Coverage enthalten die aktuellen Python-Ergebnisse. Playwright besitzt einen eigenen Rohbericht und ein eigenes Quellmanifest. Ein erster Korrekturzyklus vom 19.10. ist historisch archiviert; seine Werte werden nicht als neuer Testlauf ausgegeben.

Anhang D Persönliche Prüfergebnisse

Stand: 29.10.2026. Justin Kolar hat die praktischen Prüfungen anhand der Aufgaben P1–P6 durchgeführt und seine Ergebnisse im Chat zurückgemeldet. Die Reflexion wurde KI-gestützt formuliert, von ihm persönlich geprüft, überarbeitet und als aktuell gespeicherte Endfassung ausdrücklich freigegeben.

P1 Raumverwaltung und Kapazität

Der bereitgestellte Screenshot zeigt Prüfraum Damian mit sechs Plätzen und Standort Eigene Prüfung. Der Start wurde positiv zurückgemeldet. Der Studierende erklärte, dass eine Buchung für sieben Personen wegen der überschrittenen Kapazität eine Meldung auslösen soll. Als Erweiterungsvorschlag formulierte er die Empfehlung eines anderen Raums mit ausreichender Kapazität.

P2 Kurzer Termin, Überschneidung und Anschluss

Für die vorgegebenen Versuche 10:00–10:15, 10:00–10:30 und 10:15–10:30 am 30.10.2026 meldete er in dieser Reihenfolge „funktioniert“, „fehler“, „funktioniert“. Die konkrete Meldung lautete: „Der Raum ist von 10:00 bis 10:15 bereits gebucht. Bitte eine andere Zeit oder einen anderen Raum wählen.“ Direkt anschließende Gespräche nannte er als praktischen Nutzen. Die gespeicherte Endzeit wurde im Dialog fachlich präzisiert und die Formulierung in der persönlich überarbeiteten Reflexion freigegeben.

P3 Änderung, Stornierung und Neustart

Den angeleiteten Ablauf bestätigte er zusammenfassend mit „ist alles richtig“. Der Sollzustand war: erster Termin auf 10:30–10:45 verschoben und anschließend storniert, in der Historie auch nach Neustart erhalten; der Anschlusstermin 10:15–10:30 bleibt aktiv. Anschließend erklärte er, dass Stornieren die gespeicherte Buchung erhält, und schlug eine spätere Wiederherstellungsfunktion vor.

P4 Zwei offene Raumdialoge

Die zurückgemeldete Meldung lautete: „Der Raum wurde zwischenzeitlich geändert. Bitte den Dialog schließen, die Seite neu laden und erneut versuchen.“ Er bestätigte den Ablauf allgemein und benannte das mögliche Überschreiben durch den älteren Dialog als Problem.

P5 Persönlich gestarteter automatisierter Testlauf

Der Studierende führte .\.venv\Scripts\python.exe tools\verify.py aus. Sein PowerShell-Screenshot zeigt 118 bestandene Tests in 2,12 Sekunden sowie gerundet 99 Prozent Python-Abdeckung. Der Lauf einschließlich Rohberichten und Quellmanifest ist separat unter docs/nachweise/P5_Nutzerlauf_2026-10-29 gesichert. Der Raum-Regressionsfall war im Gesamtlauf enthalten. Die zusätzliche persönliche Durchsicht und Erklärung dieses Testcodes wurde auf Wunsch ausgelassen; eine vollständige eigenständige Code- oder Quellenprüfung wird nicht ausgewiesen.

P6 Reflexion und Freigabe

Der Studierende bewertete die Oberfläche als benutzerfreundlich und berichtete keine Unklarheiten oder Überraschungen. Die von ihm selbst überarbeitete Reflexion wird auf der Folgeseite wortgleich übernommen. Freigabedatum: 29.10.2026. Der dokumentierte Prüfumfang bleibt erhalten: Für P3 und den Datensatzstand nach P4 liegen allgemeine Bestätigungen vor, keine einzeln zurückgemeldeten Feldwerte oder persönlich abgelesenen HTTP-Statuscodes. Originalantworten, Belege und Freigabestatus stehen in docs/persoenliche_ergebnisse.json.

Anhang D Persönliche Reflexion

Stand: 29.10.2026. Freigegebene Endfassung: KI-gestützt formuliert, von Justin Kolar persönlich geprüft und überarbeitet. Die Freigabe der aktuell gespeicherten Fassung wurde im Chat ausdrücklich erteilt.

Freigegebene Reflexion

Ich habe Raumzeit anhand der angeleiteten Bedienaufgaben persönlich ausprobiert. Dabei habe ich einen Raum angelegt und die Buchung eines kurzen Termins, eine Überschneidung sowie einen direkt anschließenden Termin geprüft. Die Überschneidung wurde mit dem Hinweis auf den bereits gebuchten Zeitraum abgelehnt. Anschließende Termine sind für direkt aufeinanderfolgende Gespräche praktisch. Maßgeblich ist dabei die gespeicherte Endzeit: Endet eine Buchung um 10:15 Uhr, kann die nächste um 10:15 Uhr beginnen.

Auch Änderung, Stornierung und den gespeicherten Stand nach einem Neustart habe ich anhand der Anleitung geprüft. Beim Versuch mit zwei geöffneten Raumdialogen erschien im älteren Dialog die Meldung, dass der Raum zwischenzeitlich geändert worden war. Dieser Schutz ist erforderlich, damit ältere Eingaben neuere Änderungen nicht überschreiben.

Bei einer Stornierung bleibt die Buchung gespeichert. Das schafft eine Grundlage für eine mögliche spätere Wiederherstellungsfunktion; diese ist in Raumzeit noch nicht umgesetzt. Eine gelöschte Buchung stünde dafür nicht mehr zur Verfügung. Als weitere Verbesserung habe ich überlegt, bei unzureichender Kapazität einen passenden anderen Raum zu empfehlen. Beim Ausprobieren war für mich nichts unklar oder überraschend. Die Oberfläche empfand ich als benutzerfreundlich.

Zusätzlich habe ich die vorhandene automatisierte Python-Testsuite selbst gestartet; 118 Tests bestanden. Implementierung, Testentwicklung und Dokumentenentwürfe wurden vom KI-Assistenten erstellt und geprüft. Mein persönlicher Beitrag zu dieser Prüfung waren die beschriebenen Bedienkontrollen, Rückmeldungen und der selbst gestartete Testlauf.

Nachweisgrundlage und Grenzen

Die tatsächlichen Originalantworten und Screenshots sind in docs/persoenliche_ergebnisse.json und docs/nachweise dokumentiert. Für P3 und den Datensatzstand nach P4 liegt eine allgemeine Bestätigung vor, keine einzeln zurückgemeldeten Feldwerte.

Anhang D Persönliche Nachweise

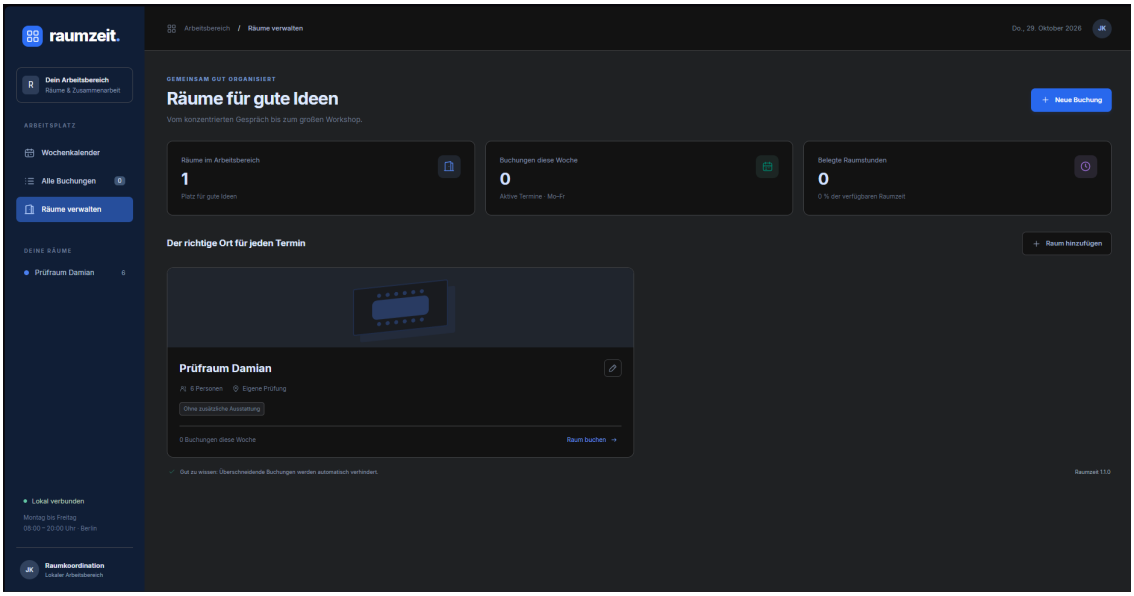


Abbildung 8: Vom Studierenden bereitgestellter Screenshot der Raumverwaltung: Prüfraum Damian mit sechs Plätzen

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\jkola> cd "D:\Uni und Arbeit\Software Engineering\Raumplaner"
PS D:\Uni und Arbeit\Software Engineering\Raumplaner> .\.venv\Scripts\python.exe tools\verify.py
.......................................................................... [ 61%]
.........................................                                  [100%]
118 passed in 2.12s
Wrote JSON report to D:\Uni und Arbeit\Software Engineering\Raumplaner\output\tests\coverage.json
Name                     Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------
raumzeit\__init__.py         1      0      0      0   100%
raumzeit\application.py      64     0      20     0   100%
raumzeit\demo.py             13     0      4      0   100%
raumzeit\domain.py           91     0      22     0   100%
raumzeit\ports.py            8      0      0      0   100%
raumzeit\storage.py          98     3      16     0    97%   137-139
raumzeit\web.py              95     0      14     1    99%   21->23
--------------------------------------------------------------------
TOTAL                    370      3     76      1    99%

PS D:\Uni und Arbeit\Software Engineering\Raumplaner>
```

Abbildung 9: Vom Studierenden gestarteter Python-Testlauf: 118 bestanden; Python-Abdeckung gerundet 99 Prozent

Diese Screenshots wurden vom Studierenden im Chat übermittelt. Der persönliche Python-Lauf bleibt einschließlich Rohberichten separat unter docs/nachweise/P5_Nutzerlauf_2026-10-29 erhalten. Die Browserbilder in Anhang C stammen aus den automatisierten KI-gestützten Prüfungen.